

UNIVERSIDAD ANÁHUAC
QUERÉTARO

PROYECTO FINAL: ESTRUCTURA DE DATOS Y ALGORITMOS

Hito Intermedio: Avance de Implementación y Control de Cronograma

Alumno: Noé Emmanuel Alvarado Ríos

Fecha: 4 de noviembre de 2026

Tema: Triangulación de Delaunay para Interpolación de Alturas

Enlace al repositorio Git:
https://github.com/EmmanuelAlvarado-ai/Triangulacion-de-Delanouy

1. Auditoría y Análisis del Cronograma (Fase 1 vs. Realidad)

A continuación, se presenta la tabla comparativa de los hitos comprometidos en el protocolo original (Fase 1) frente al progreso real hasta la fecha de este reporte intermedio.

Hito / Actividad (Fase 1 vs. Ajuste)	**Semanas Planeadas**	**Fechas Planeadas**	**Fecha Real / Cierre**	**Estado**
Hito 1: Clases base (Point, Triangle) y predicado in_circle con módulo decimal.	Sem 1 - 2	21 Sep – 04 Oct	04 Oct	Completado
Hito 1.5: Línea base por Fuerza Bruta $O(n^4)$ y suite de pruebas test_*.py.	N/A (Hito Int.)	05 Oct – 15 Oct	28 Oct	Completado
Hito 2: Súper-triángulo, inserción incremental y legalize_edge (Edge Flip).	Sem 3 - 4	05 Oct – 18 Oct	29 Oct	Completado
Hito 3: Estructura de localización DAG $O(\log n)$ y remoción de súper-triángulo.	Sem 5 - 6	19 Oct – 08 Nov	En curso	En progreso
Hito 4: Benchmarks ($n = 10^2$ a 10^4) y	Sem 6+	09 Nov – Entrega	Pendiente	En tiempo

visualización interactiva.				

Justificación técnica de desfases y Plan de Contingencia: Durante las semanas 3 y 4 se presentó un ligero ajuste de tiempos debido a dos factores técnicos principales:

1. La necesidad de incorporar y probar un algoritmo exhaustivo de Fuerza Bruta $O(n^4)$ no contemplado inicialmente, pero necesario para la validación cruzada solicitada en este hito intermedio.
2. La complejidad computacional de mantener la consistencia bidireccional en los punteros de los objetos Triangle al ejecutar la subrutina recursiva legalize_edge, así como el ajuste de los vértices a una orientación *counter-clockwise* (CCW) para asegurar la precisión matemática del determinante.

Plan de contingencia: Actualmente, el algoritmo óptimo ya es 100% funcional localizando el triángulo contenedor mediante búsqueda lineal temporal. Esto garantiza que las estructuras de datos y el "edge flip" operan sin errores. Durante la primera semana de noviembre se sustituirá la búsqueda lineal por la inserción en el Grafo Acíclico Dirigido (DAG), manteniendo intacta la lógica geométrica ya probada, asegurando así la entrega final en tiempo y forma.

2. Arquitectura de Clases e Implementación en Python

El proyecto ha sido modularizado para separar la lógica geométrica pura, la estructura de datos del algoritmo, y la validación. La arquitectura actual se compone de los siguientes módulos:

- **models.py (Capa de Geometría Robusta):**
  - **Clase Point:** Almacena las coordenadas espaciales (x, y, z). Utiliza internamente el módulo decimal de Python para evitar errores de punto flotante en las evaluaciones geométricas críticas.
  - **Clase Triangle:** Mantiene referencias directas a sus 3 vértices (Point) y a sus 3 vecinos adyacentes (Triangle). Esta estructura de punteros permite realizar las actualizaciones topológicas en tiempo $O(1)$ constante. Incluye un atributo booleano is_active indispensable para mantener el historial vivo en el futuro DAG.
  - **Subrutinas Matemáticas:** orient2d (para garantizar orden antihorario) e in_circle (determinante matemático para evaluar la Propiedad de Delaunay).
- **delaunay.py (Capa de Algoritmos):**
  - **delaunay_brute_force:** Función que evalúa las $\binom{n}{3}$ combinaciones posibles, entregando la triangulación en tiempo $O(n^4)$. Sirve como fuente de la verdad.
  - **Clase IncrementalDelaunay:** Contiene la lógica del algoritmo Randomized Incremental Construction (RIC). Maneja la inicialización del súper-triángulo imaginario, la división de la malla (insert_point) y la recursividad para restaurar la validez de Delaunay localmente (legalize_edge).
- **test_delaunay.py y main_visual.py (Capa de Pruebas y Presentación):**

- Pruebas unitarias que aseguran que el resultado de la estructura incremental sea matemáticamente idéntico al de la fuerza bruta.
- Módulo de renderizado con Matplotlib para la visualización del relieve 3D interactivo y la construcción paso a paso.

3. Estrategia de Datos para el Benchmark Final

Para la entrega de noviembre, el análisis de rendimiento empírico (Benchmark) no se realizará con puntos aleatorios simples, sino que se diseñará un protocolo experimental escalonado basado en la naturaleza de los Sistemas de Información Geográfica (SIG):

Nivel 1: Validación de Exactitud Lógica ($n \in \{10, 25, 50\}$**)** Se correrán instancias pequeñas simultáneamente en delaunay_brute_force y en el modelo óptimo. El test pasará estrictamente si ambos conjuntos de triángulos resultantes son idénticamente iguales (comparación por vértices).

Nivel 2: Escalamiento Asintótico ($n \in \{10^2, 5 \times 10^2, 10^3, 5 \times 10^3, 10^4\}$**)** Se generarán lotes de "terrenos sintéticos". Los puntos se distribuirán uniformemente en el plano XY, pero su altura Z estará dictada por una función de ruido continuo (ej. combinaciones de funciones trigonométricas). Para prevenir escenarios de degeneración geométrica estricta (4 puntos co-circulares), las coordenadas recibirán una perturbación microscópica $\epsilon \approx \pm 10^{-7}$.

Nivel 3: Recolección de Métricas Analíticas Durante el escalamiento del Nivel 2, el sistema registrará internamente:

1. **Tiempo de CPU (segundos):** Para demostrar que la curva de tiempo de la implementación DAG se ajusta empíricamente a la teórica $O(n \log n)$.
2. **Frecuencia de Legalización:** Promedio de llamadas a legalize_edge (Edge Flips) por cada punto insertado, con el objetivo de demostrar la hipótesis de que la reestructuración local permanece constante independientemente del tamaño de la muestra.
3. **Profundidad de Búsqueda:** Cantidad de saltos promedio en el historial de triángulos desactivados, evidenciando la eficiencia del DAG vs la búsqueda lineal temporal.